软件工程课程 · 计算机班

Lab1 + Lab2 讲评

2026年5月25日

PART I

Lab1 讲评

所有小组都过线，分差不大

97 / 105 **总分中位数**

9 分 **总分差小**

8.0 / 10 **面试均分**

分数分布（含面试，满分 105）

区间	组数
92 – 94 (低梯队)	3
95 – 97 (中梯队)	10
98 – 101 (高梯队)	8

功能需求：三项基础需求100% 满分，理解完整

01

用户注册

21 组全部正确实现

02

数据隔离

不同用户任务隔离，21 组全部满分

03

错误提示

前后端能告知失败原因，21 组全部满分

普遍没做好的三件事

得分率 69.5%

工程成熟度

13 组 < 70%

- 面试中现场答辩反映不出对代码的掌控力
- 演示时容易卡住、依赖某一位组员
- 建议：能跑、能解释，比报告写多长更重要

得分率 72.4%

Commit Message 规范

仅 1 组满分

- 缺 type/scope 前缀（feat / fix / docs / refactor）
- 一次 commit 范围过大，把一周工作塞一条
- 推荐：Conventional Commits 规范

<type>[optional scope]: <description>

[optional body]

得分率 90.0%

数据库设计

0 组满分

- 字段命名 / 类型 / 约束普遍有保留
- 缺时间字段（created_at / updated_at）
- 所有权字段位置不规范

面试中的3 类提问

类别 A · 几乎每组都问

技术选型为什么是这样

你们用了 xxx技术栈：

- 为什么选这个？
- 有对比过不同的技术栈吗？

类别 B · 挑组追问

设计取舍为什么是这样

你给系统加什么，就得能解释为什么这个东西值得加：

- 过度拓展的：
  - 为未来预留了很多字段，但这种多余设计很可能是负担，你们怎么考虑的？
- 设计权衡：
  - 为什么把「剔除发起人自身」放在前端做，而不是后端做？

类别 C · 少数组

通用追问

- 任务删除是硬删还是软删？
- 为什么？
- 考虑过可拓展性吗？

功能拓展：加什么、不加什么——从用户体验出发

Lab1 班级里出现过的「额外功能」

任务能力	软删除 · 子任务 · 任务依赖关系 · 批量删除 · 搜索 / 筛选 / 排序 / 分页
视图扩展	看板视图 · 日程视图 · 自定义标签
用户体验	深色模式 · 智能自然语言录入 · 待办预警 · 邮箱动态验证码
工程闭环	操作审计 · i18n（多语言） · 部署上线 · 提前做团队协作
个人化	自定义头像 · 密码修改 · 用户注销

加之前，先问两件事

1 这个功能从用户体验角度，真的有必要吗？

- 用户真的会用、会受益吗？还是只是技术上酷？
- 它会不会让真正核心的功能（注册/任务/隔离）反被稀释？

2 你能长期解释和维护这个设计吗？

- 知道什么不该写
- 不该过度设计
- 这是会伴随你们整个职业生涯的能力，比多写一个功能更重要

之后Lab，大家只需要做「需求文档中的要求」。
但「从用户视角判断该不该做」和「能否长期解释维护」这两个问题，**每个 Lab、每一行代码、未来每个项目都还会问你。**

PART II

Lab2 讲评

得分开始分化

75 / 100 **总分中位数**

16 分 **分差较大**

70 / 100 **Lab1顶尖组的表现**

分数分布

区间	组数
67 – 70 最低梯队	5
71 – 75 中下	6
76 – 80 中上	6
81 – 83 最高梯队	4

做对的地方

100% 21/21 用户能够创建团队	100% 21/21 Admin ↔ Member 双向切换	100% 21/21 显示用户在团队中的角色	100% 21/21 Commit Message 规范
95%+ 20/21 展示团队成员及权限	95%+ 20/21 个人任务列表融合	95%+ 19/21 Owner 添加成员	98%+ 分支合并请求 / AI 使用记录

功能实现中缺失对于边界情况的考虑

2 分

重复添加成员

0 分	满分
7 个组	9 个组

现象

需求第 3 节明确提示「如何避免重复数据或非法操作？」

可行方案

service 层 + DB unique 双保险，返回明确错误（「该用户已在团队中」），不是 500。

3 分

团队名称重复

0 分	满分
6 个组	10 个组

现象

和重复加成员类似

可行方案

创建时查重 + 返回业务级错误码；不要让用户自己点两次发现问题。

2 分

任务列表分页

0 分	满分
10 个组	9 个组

现象

在任务数据量变大时，如何进行展示？

可行方案

列表类接口默认带 limit+offset 或游标。真实任务列表会有几百几千条。

数据隔离：班级最普遍的功能性扣分

0

个组拿到 6/6 满分

均分 4.1 / 满分 6 · 21 组全部部分扣分

普遍出现的不足：

1. **任务查询只按 status / boardId 过滤，没绑 team_id**

   → 知道 ID 就能跨团队读

2. **个人任务列表混入了其他团队的任务**

   → team_id 没在 query 层做硬过滤

3. **删除/修改只查「任务存在」，没查「你是不是这个团队的成员」**

   → 非成员也能改

推荐的方案：

- Group10（FastAPI dependency 复用权限校验）和 Group15 是仅有的 2 组在「后端强制校验权限」项拿满分。
- 数据隔离的本质：每个查询都要对 (资源 ID + 用户/团队 ID) 双过滤，不是「用了 JWT 就有」。

关键设计决策：5 分项均分 2.9，全班无满分

问题 大家对于「关键设计决策」的描述基本是「我们用了 X 技术」。这只是陈述事实，不是记录关键决策。

真正的「关键设计决策」应有四要素

1 当时的问题

你面临什么具体的技术或业务难题？

2 候选方案

至少列出 2 个考虑过但没选的方案。

3 选择理由

为什么是这个？性能 / 成本 / 团队熟悉度 / 可维护性？

4 引入的新成本

这个选择会带来什么新限制？以后想换难不难？

推荐阅读：https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

PART III

工程质量

页面或入口文件承担过多职责

功能风险

团队权限和任务状态扩展时，多种交互状态会相互影响。后续 Lab 加功能时改一处崩三处。

要点

- 8 个组都存在这个问题
- 大文件本身不是罪过，但同时处理 UI + 请求 + 权限 + 筛选 + 弹窗 + 数据转换时，扩展性差
- 优化方案：
  - 拆成工具栏、列表、表单弹窗、详情弹窗、操作 composable

// 案例 · 单文件 1462 行的任务管理组件 (Vue)

```
<div class="toolbar">
  <input v-model="searchKeyword" @input="handleSearch" />
  <button @click="toggleCollapseAll">
    {{ isAllCollapsed ? '展开全部' : '折叠全部' }}
  </button>
  <div class="filters">
    <button
      v-for="status in filterOptions"
      :class="{ active: filterStatus === status.value }"
      @click="setFilterStatus(status.value)"
    >
      {{ status.label }}
    </button>
  </div>
  <button v-if="props.showCreateButton" @click="openCreateModal">
    + 创建任务
  </button>
</div>

// 同一文件还包含：搜索、折叠、筛选、删除、详情、创建入口
```

请求失败被隐藏，用户看到的不是系统真实状态

功能风险

权限错误、接口错误或网络失败都会表现成「任务为空」。用户以为「没有数据」，而不是「系统失败」。

要点

- 每次出现都是用户体验灾难
- 核心数据加载失败时不应静默转为空列表
- 优化方案：
  - 核心列表请求失败进入 error 状态，非核心数据才允许弱降级
  - 权限错误——提醒用户
  - 接口错误——开发者修复
  - 网络错误——提醒用户

// 案例 · 团队任务请求失败被 .catch(() => []) 转空

```
const tasksRes = await getTeamTasks(teamId, params).catch(() => []);
const tasksData = tasksRes?.items
                || tasksRes?.tasks
                || (Array.isArray(tasksRes) ? tasksRes : []);
setTasks(tasksData);

const [teamDetailRes, tasksRes, userRes] = await Promise.all([
  getTeamDetail(teamId),
  getTeamTasks(teamId, params).catch(() => []),  // ← 错误被吞
  getUserProfileApi().catch(() => null),
]);

// 用户看到"任务为空"，其实可能是 403 / 500 / 网络中断
```

模型字段和数据访问代码不一致

功能风险

任务列表、详情、更新、删除可能访问不存在字段。个人任务和团队任务边界都不可靠。

要点

- 核心功能可能直接运行失败
- 不是代码风格，是字段语义出现错位
- 优化方案：
  - 明确「创建者 / 被分配人 / 团队归属」三类字段，按业务语义过滤

// 案例 · models 与 crud 字段不一致

```
# models.py
team_id     = Column(Integer, ForeignKey("teams.id"), nullable=False)
creator_id  = Column(Integer, ForeignKey("users.id"), nullable=False)
assigned_to = Column(Integer, ForeignKey("users.id"), nullable=True)

# crud.py ← 模型里没有 user_id 字段!
def get_tasks(db, user_id, skip=0, limit=100):
    return db.query(models.Task)\
             .filter(models.Task.user_id == user_id)\
             .offset(skip).limit(limit).all()

def create_task(db, task, user_id):
    db_task = models.Task(**task.dict(), user_id=user_id)
    # ↑ 模型构造器收不到 user_id，会报错或被忽略
```

服务端输入校验和密码安全

功能风险

空用户名、空密码、明文密码进入数据库。后续登录、权限边界全部不可靠。

要点

- 注册接口不能只靠数据库约束或前端表单
- 优化方案：
  - 服务端 schema 校验 + 用户名查重 + 密码哈希 + 明确错误状态码

// 案例 · 注册接口零校验 + 明文密码

```
@auth_bp.route('/register', methods=['POST'])
def register():
    data = request.get_json()

    user = User(
        username=data.get('username', ''),  # ← 空值兜底
        password=data.get('password', '')   # ← 明文存储
    )
    db.session.add(user)
    db.session.commit()

    return jsonify({
        'code': 200,
        'message': '注册成功',
        'data': user.to_dict()
    }), 200

# 缺：schema 校验、用户名查重、密码哈希、错误状态码
```

好例子

Group1

后端服务层强制权限

backend/src/teams/teams.service.ts:80–170

Owner / Admin / Member 判断在后端 service 层集中执行，前端只是 UI。

Group10

FastAPI dependency 复用权限校验

src/core/dependencies.py:31–120

认证、团队成员、Owner/Admin 检查都通过 Depends(...) 复用，路由层干净。

Group20

统一错误映射 + 服务端日志

app/routes/team_tasks.py:30–71

内部异常记录日志，对前端返回安全的统一错误；不暴露堆栈。

Q & A

Lab1 + Lab2 讲评 · 谢谢